

NYCU DLP LAB2 Report

1. Overview

- DataSet :

We use BCI Competition IV 2a as the dataset, there are total four labels, including left hand, right hand, feet, and tongue. This dataset consists of EEG signals from several subjects, each subject contains 2 sessions. There are 288 trials in each session, 72 for each label.

- SCCNet :

This section presents the design of the proposed SCCNet and its architecture. A major part of SCCNet consists of convolutional kernels that capture the spatial and temporal characteristics of the EEG data. The design of SCCNet focuses on leveraging the benefits from applying spatial filtering to EEG data for purposes such as feature extraction and noise suppression. The architecture of SCCNet is illustrated in the below picture. The input to SCCNet is multi-channel EEG data arranged in 2-dimensions, with C channels and T time points. The architecture of SCCNet consists of four blocks: the first convolution block, the second convolution block, the pooling block, and the softmax block.

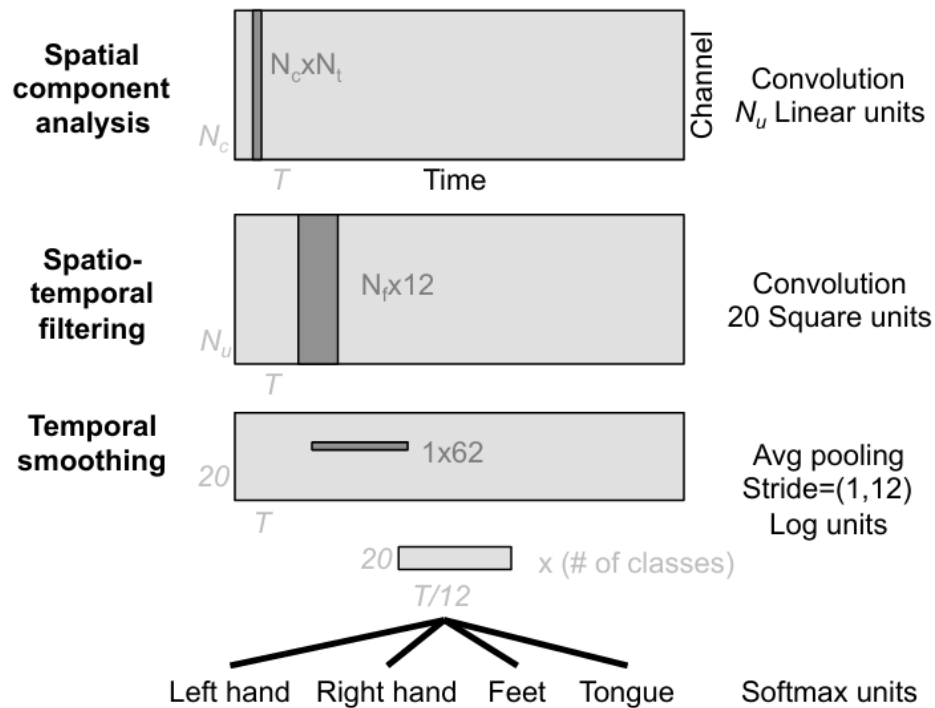


Fig. 1. The example architecture of SCCNet.

- Goal :

We implemented the SCCNet as described in the paper provided by the TA. We compared the results using three different methods: subject-dependent, leave-one-subject-out, and leave-one-subject-out with fine-tuning.

2. Implementation Details

A. Details of training and testing code

- I use Adam as our optimizer with a decay of 0.0001, as mentioned in the paper. We adopt CrossEntropyLoss as our loss function. Additionally, we record the loss to plot the learning rate curve
- I can choose the train method and mode to choose our training method , including SD, LOSO, LOSO with finetune .

- I print out the training accuracy and loss to monitor the model's performance.

```
def train(n_epochs ,models,lr,mode,method ,batch_size,model_path):
    #####Prepare Data #####
    #DataSet
    train_set = MIBCI2aDataset(mode=mode ,method=method)
    #DataLoader
    train_loader = DataLoader(train_set,batch_size=batch_size,shuffle=True)

    # "cuda" only when GPUs are available.
    device = "cuda" if torch.cuda.is_available() else "cpu"

    # Initialize a model, and put it on the device specified.
    model = models.to(device)

    #loss function
    criterion = nn.CrossEntropyLoss()

    # L2 regulation
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    #scheduler = StepLR(optimizer, step_size=50, gamma=0.9)

    # The number of training epochs.
    n_epochs = n_epochs

    softmax = nn.Softmax(dim=-1)

    train_losses = []

    for epoch in range(n_epochs):
        # These are used to record information in training.
        train_loss = []
```

```

train_accs = []

#####training#####
model.train()
for batch_idx, (features, labels) in enumerate(train_loader):
    features, labels = features.to(device), labels.to(device)

    # Gradients stored in the parameters in the previous batch
    #be cleared out first.
    optimizer.zero_grad()
    # Forward the data. (Make sure data and model are on the same device)

    logits = model(features.to(torch.float32))

    # Obtain the probability distributions by applying softmax
    probs = softmax(logits)

    # Calculate the cross-entropy loss.
    # We don't need to apply softmax before computing
    #cross-entropy as it is done automatically.
    loss = criterion(logits, labels.to(torch.int64))

    # Compute the gradients for parameters.
    loss.backward()

    # Update the parameters with computed gradients.
    optimizer.step()

    # Compute the accuracy for current batch.
    acc = (logits.argmax(dim=-1) == labels).float().mean()

    # Record the loss and accuracy.
    train_loss.append(loss.item())
    train_accs.append(acc)

```

```

# The average loss and accuracy of the training set is
#the average of the recorded values.
train_loss = sum(train_loss) / len(train_loss)
train_acc = sum(train_accs) / len(train_accs)

train_losses.append(loss.item())

# Print the information.
print(f"[ Train | {epoch + 1:03d}/{n_epochs:03d} ]"
      "loss = {train_loss:.5f}, acc = {train_acc:.5f}")
#update lr
#scheduler.step()

#Save the model
torch.save(model, model_path)
print(f'model save to {model_path}')

plot_learning(n_epochs,train_losses)

```

```

def test(mode,method,batch_size,model_path):
    # "cuda" only when GPUs are available.
    device = "cuda" if torch.cuda.is_available() else "cpu"
    ##### testing set#####

    #load the testing set
    test_set = MIBCI2aDataset(mode=mode,method=method)
    testloader = DataLoader(test_set,batch_size=batch_size ,shuffle=True)

    # loss function
    criterion = nn.CrossEntropyLoss()
    #load the model
    model = torch.load(model_path)
    #define softmax

```

```

softmax = nn.Softmax(dim=-1)
# turn the model into eval mode
model.eval()

test_loss = 0
correct = 0
predictions = []
true_labels = []
total=0

with torch.no_grad():
    for features, labels in testloader:
        # transfer to GPU
        features, labels = features.to(device), labels.to(device)

        # Forward pass
        logits = model(features.to(torch.float32))
        preds =softmax(logits)
        # Calculate loss
        loss = criterion(logits, labels.to(torch.int64))
        test_loss += loss.item() # Accumulate the total test loss

        # Get predicted labels
        preds = logits.argmax(dim=1)
        predictions.extend(preds.cpu().numpy()) # Store predicted labels
        true_labels.extend(labels.cpu().numpy()) # Store true labels
        total+=labels.size(0)
        # Calculate accuracy
        correct += (preds == labels).sum().item()

# Average test loss
avg_test_loss = test_loss / len(testloader) #8

# Accuracy calculation
accuracy = correct / total

```

```
print(f"Average Test Loss: {avg_test_loss:.4f}")
print(f"Accuracy: {accuracy *100:.2f}%")
```

B. Details of the SCCNet

- According to the paper, we implement the SCCNet, which consists of four layers.
- The initial layer uses a CNN to extract EEG features. This layer mimics a spatial component analysis by decomposing the original EEG data from the channel domain into a component domain, employing N_u filters with a kernel size of (C, N_t) .
- Since the multi-channel EEG data are transferred into multiple EEG spatial components, the size of channel domain becomes 1 and the size of component domain becomes the number of convolutional kernels, N_u . Thus, we do some permutation
- The second convolution uses N_c convolutional kernels with a size of $(N_u, 12)$, where the number 12 corresponds to 0.1 seconds along time domain. At this step, the convolution procedure applies to both the temporal and spatial component domains. The spatial-temporal convolution is expected to perform spectral filtering, inter-component correlation and other spatial-temporal analysis on the EEG spatial components. Zero-padding and batch normalization were applied to both the first and second convolutions, as well as 2 regularization with a coefficient of 0.0001. We use square activation to extract the power from the data, as spectral power change is the most prominent marker in MI EEG. Next, we applied dropout with a rate of 0.5 to prevent overfitting.
- Following the two convolutional blocks, we apply an average pooling layer with a size of $(1, 62)$ to perform smoothing in the temporal domain and reduce the dimensions. The number 62 corresponds to 0.5 seconds along the time domain. We use a stride of $(1, 12)$. After pooling, we apply a log layer to improve not only numerical stability but also convergence.

- In the final layer, we apply a flatten operation to prepare the data for the subsequent fully-connected layer. During training, we apply softmax to obtain the probabilities for the four classes.

```
class SquareLayer(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        return x**2

class logLayer(nn.Module):
    def __init__(self):
        super().__init__()
    def forward(self, x):
        return torch.log(x)

# 定義 SCCNet 類別
class SCCNet(nn.Module):

    def __init__(self, numClasses=4, timeSample=438,
                 Nu=22, C=22, Nc=20, Nt=16, dropoutRate=0.5):
        super(SCCNet, self).__init__()

        self.conv1 = nn.Conv2d(1, Nu, (C, Nt), padding=0)
        #(batch size ,kernel ,height ,width)
        self.bn1 = nn.BatchNorm2d(Nu)

        self.conv2 = nn.Conv2d(1, Nc, (Nu, 12), padding=(0,6))
        self.bn2 = nn.BatchNorm2d(Nc)

        # square activation function
```

```

self.square = SquareLayer()
# dropout
self.dropout = nn.Dropout(dropoutRate)
# pooling (use average pooling)

self.pool = nn.AvgPool2d((1, 64), stride=(1, 12))
self.logLayer = LogLayer()
# output shape (20,T/12)
afterPoolSize = (timeSample-64)//12
self.fc = nn.Linear(Nc*afterPoolSize, numClasses) #620

def forward(self, x):
    #print("Input shape:", x.shape)
    x = x.unsqueeze(1) # (batch , 1 ,22,438)
    #print("After unsqueeze:", x.shape)

    #####first layer#####
    x = self.conv1(x) # (batch,22,1,423) width -Nt+1
    #print("After conv1:", x.shape)
    x = self.bn1(x)
    #print("After bn1:", x.shape)
    x = self.dropout(x)
    x = x.permute(0,2, 1, 3) # [batch, 1, 22, 423]

    #####Second layer#####
    #print("After permute:", x.shape)
    x = self.conv2(x) #[batch, 20, 1, 424]
    #print("After conv2:", x.shape)
    x = self.bn2(x) #[32, 20, 1, 424]
    #print("After bn2:", x.shape)
    x = self.square(x) # [32, 20, 1, 424]
    #print("After square2:", x.shape)
    x = self.dropout(x) # [32, 20, 1, 31]

```

```

#print("After dropout:", x.shape)
x = x.permute(0,2, 1, 3)

#####third layer#####
x = self.pool(x) # [32, 1, 20, 31]
#print("After pool:", x.shape)
x=self.logLayer(x)

#####fourth layer#####
x = x.flatten(1)    #[32, 620]
#print("After flatten:", x.shape)
x = self.fc(x) #[32, 4]
#print("After fc:", x.shape)

return x

```

3. Analyze on the experiment results

A. Discover during the training process

- The Leave-One-Subject-Out (LOSO) method performs better than the Subject-Dependent (SD) method.
 - Reason :
 - **Generalization:**
 - **Issue:** The LOSO method trains the model on multiple subjects and tests it on one held-out subject, leading to better generalization across different subjects.
 - **Reason:** Training on diverse data helps the model learn more robust features that generalize better to unseen subjects.
 - **Reduced Overfitting:**

- **Issue:** The SD method might overfit to the specific characteristics of a single subject's data, resulting in poorer performance on other subjects.
- **Reason:** LOSO reduces the risk of overfitting by exposing the model to a wider variety of data during training.
- **Data Variability:**
 - **Issue:** LOSO captures more variability in the EEG data since it includes data from multiple subjects.
 - **Reason:** This increased variability helps the model become more adaptable and perform better on new subjects.
- I tried different learning rates, including 0.1, 0.01, 0.001, and 0.0001. The learning rate of 0.001 performed the best.
 - The high learning rate results in overly large parameter updates, causing the model to overshoot the optimal solution.
 - The low learning rate leads to very small parameter updates, making the training process extremely slow and possibly terminating before the model achieves optimal performance.
- Overfitting: The model performs better during training but worse during testing.
- The strategy of learning rate scheduling will not ha

B. Comparison between the three training methods

Subject-Dependent (SD) Method:

- **Advantages:** Training and testing are conducted on the same subject's data, allowing the model to adapt well to the specific characteristics of that subject.
- **Disadvantages:** The model's generalization ability is poor, and it may perform poorly on other subjects.

Leave-One-Subject-Out (LOSO) Method:

- **Advantages:** Training the model on data from multiple subjects improves the model's generalization ability.

- **Disadvantages:** Training time is longer, and it consumes more computational resources.

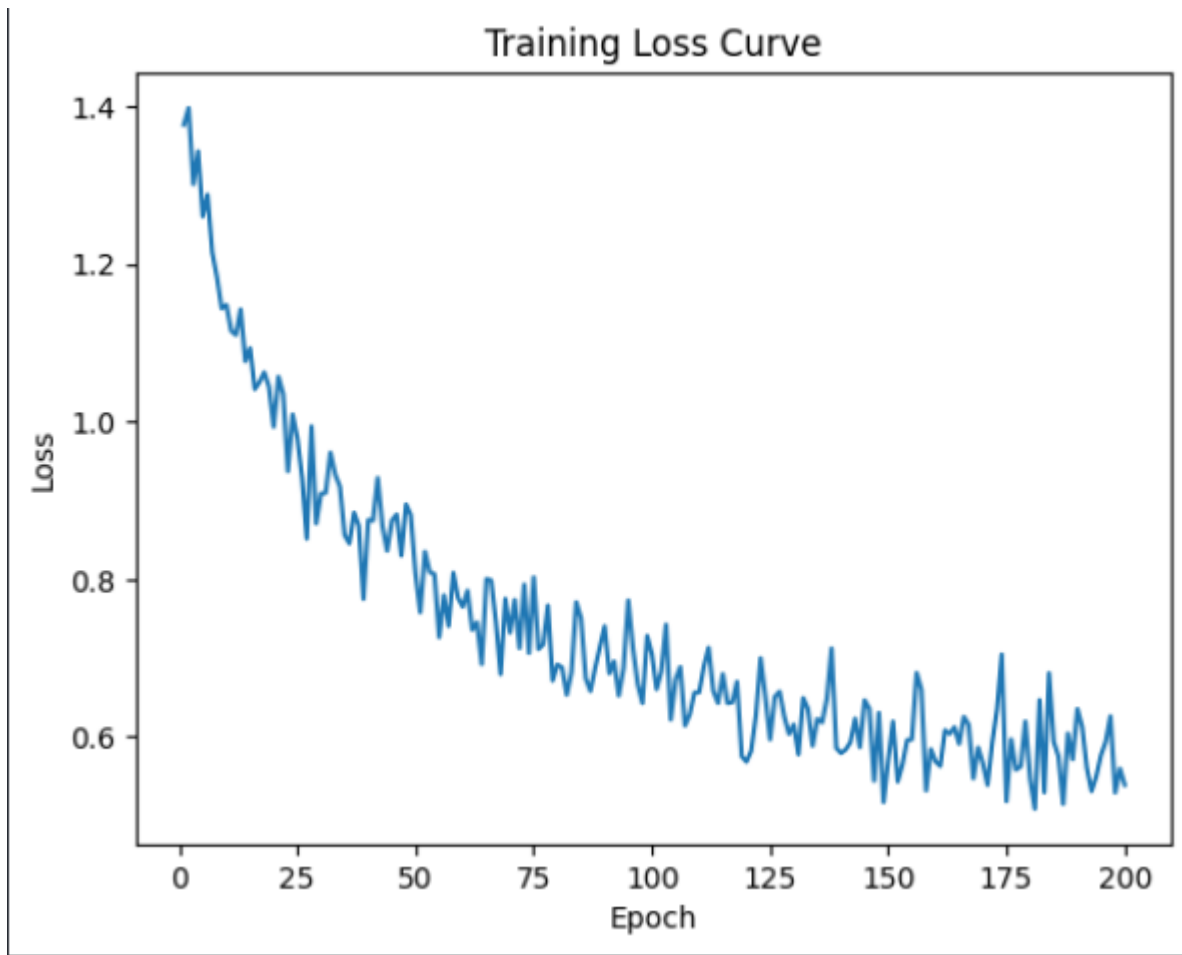
LOSO with Fine-Tuning Method:

- **Advantages:** The model is first trained on data from multiple subjects and then fine-tuned on data from a specific subject, combining the advantages of both methods.
- **Disadvantages:** Requires more training steps and resources.
- The accuracy:

```
The method is SD
Average Test Loss: 1.0975
Accuracy: 60.11%
The method is LOSO
Average Test Loss: 1.0573
Accuracy: 63.19%
The method is LOSO with finetune
Average Test Loss: 1.1405
Accuracy: 80.21%
```

-
- The hyperparameter
 - lr:1e-3
 - batch_size:288
 - L2-regulation with : 1e-4

Subject dependent (SD)



#SD

```
[ Train | 001/200 ] loss = 1.39534, acc = 0.28342
[ Train | 002/200 ] loss = 1.36893, acc = 0.31641
[ Train | 003/200 ] loss = 1.32675, acc = 0.36806
[ Train | 004/200 ] loss = 1.30754, acc = 0.38845
[ Train | 005/200 ] loss = 1.27387, acc = 0.40929
[ Train | 006/200 ] loss = 1.25004, acc = 0.43663
[ Train | 007/200 ] loss = 1.22591, acc = 0.45964
[ Train | 008/200 ] loss = 1.19328, acc = 0.46528
[ Train | 009/200 ] loss = 1.16226, acc = 0.47830
[ Train | 010/200 ] loss = 1.14345, acc = 0.49219
[ Train | 011/200 ] loss = 1.12770, acc = 0.50651
[ Train | 012/200 ] loss = 1.10264, acc = 0.52691
[ Train | 013/200 ] loss = 1.09138, acc = 0.52474
```

```
[ Train | 014/200 ] loss = 1.06323, acc = 0.54297
[ Train | 015/200 ] loss = 1.05936, acc = 0.54210
[ Train | 016/200 ] loss = 1.04378, acc = 0.55686
[ Train | 017/200 ] loss = 1.02827, acc = 0.56163
[ Train | 018/200 ] loss = 1.01878, acc = 0.56380
[ Train | 019/200 ] loss = 1.01588, acc = 0.55295
[ Train | 020/200 ] loss = 0.99737, acc = 0.57595
[ Train | 021/200 ] loss = 0.99376, acc = 0.57812
[ Train | 022/200 ] loss = 0.97335, acc = 0.58898
[ Train | 023/200 ] loss = 0.95777, acc = 0.59158
[ Train | 024/200 ] loss = 0.95932, acc = 0.59418
[ Train | 025/200 ] loss = 0.93614, acc = 0.61111
[ Train | 026/200 ] loss = 0.92901, acc = 0.61328
[ Train | 027/200 ] loss = 0.90996, acc = 0.61979
[ Train | 028/200 ] loss = 0.92306, acc = 0.61892
[ Train | 029/200 ] loss = 0.92151, acc = 0.61328
[ Train | 030/200 ] loss = 0.91393, acc = 0.61806
[ Train | 031/200 ] loss = 0.89666, acc = 0.63585
[ Train | 032/200 ] loss = 0.89745, acc = 0.62370
[ Train | 033/200 ] loss = 0.89544, acc = 0.63715
[ Train | 034/200 ] loss = 0.90964, acc = 0.61849
[ Train | 035/200 ] loss = 0.87877, acc = 0.63759
[ Train | 036/200 ] loss = 0.86976, acc = 0.64627
[ Train | 037/200 ] loss = 0.86680, acc = 0.64149
[ Train | 038/200 ] loss = 0.85971, acc = 0.65061
[ Train | 039/200 ] loss = 0.84449, acc = 0.65972
[ Train | 040/200 ] loss = 0.84551, acc = 0.65842
[ Train | 041/200 ] loss = 0.84838, acc = 0.66233
[ Train | 042/200 ] loss = 0.84444, acc = 0.65321
[ Train | 043/200 ] loss = 0.85782, acc = 0.65148
[ Train | 044/200 ] loss = 0.82385, acc = 0.66710
[ Train | 045/200 ] loss = 0.83967, acc = 0.65712
[ Train | 046/200 ] loss = 0.82074, acc = 0.67491
[ Train | 047/200 ] loss = 0.80766, acc = 0.66970
[ Train | 048/200 ] loss = 0.83212, acc = 0.65191
[ Train | 049/200 ] loss = 0.82376, acc = 0.65017
```

```
[ Train | 050/200 ] loss = 0.81521, acc = 0.67318
[ Train | 051/200 ] loss = 0.80163, acc = 0.67318
[ Train | 052/200 ] loss = 0.79700, acc = 0.67795
[ Train | 053/200 ] loss = 0.79124, acc = 0.67665
[ Train | 054/200 ] loss = 0.77253, acc = 0.67839
[ Train | 055/200 ] loss = 0.76524, acc = 0.70009
[ Train | 056/200 ] loss = 0.78132, acc = 0.68707
[ Train | 057/200 ] loss = 0.75182, acc = 0.70269
[ Train | 058/200 ] loss = 0.77771, acc = 0.68576
[ Train | 059/200 ] loss = 0.77963, acc = 0.67405
[ Train | 060/200 ] loss = 0.79155, acc = 0.67752
[ Train | 061/200 ] loss = 0.76530, acc = 0.69878
[ Train | 062/200 ] loss = 0.74262, acc = 0.71224
[ Train | 063/200 ] loss = 0.74007, acc = 0.71007
[ Train | 064/200 ] loss = 0.74964, acc = 0.69792
[ Train | 065/200 ] loss = 0.73508, acc = 0.71398
[ Train | 066/200 ] loss = 0.75142, acc = 0.70009
[ Train | 067/200 ] loss = 0.74143, acc = 0.69922
[ Train | 068/200 ] loss = 0.76332, acc = 0.69010
[ Train | 069/200 ] loss = 0.74853, acc = 0.70616
[ Train | 070/200 ] loss = 0.75733, acc = 0.68620
[ Train | 071/200 ] loss = 0.72305, acc = 0.71354
[ Train | 072/200 ] loss = 0.72301, acc = 0.71181
[ Train | 073/200 ] loss = 0.73865, acc = 0.70095
[ Train | 074/200 ] loss = 0.72154, acc = 0.71701
[ Train | 075/200 ] loss = 0.74356, acc = 0.69661
[ Train | 076/200 ] loss = 0.72993, acc = 0.70052
[ Train | 077/200 ] loss = 0.71177, acc = 0.70616
[ Train | 078/200 ] loss = 0.71511, acc = 0.71484
[ Train | 079/200 ] loss = 0.69136, acc = 0.72483
[ Train | 080/200 ] loss = 0.70825, acc = 0.71918
[ Train | 081/200 ] loss = 0.70847, acc = 0.72396
[ Train | 082/200 ] loss = 0.70829, acc = 0.71528
[ Train | 083/200 ] loss = 0.67464, acc = 0.74132
[ Train | 084/200 ] loss = 0.71040, acc = 0.70790
[ Train | 085/200 ] loss = 0.69794, acc = 0.72135
```

```
[ Train | 086/200 ] loss = 0.69631, acc = 0.72266
[ Train | 087/200 ] loss = 0.69437, acc = 0.72960
[ Train | 088/200 ] loss = 0.68883, acc = 0.73090
[ Train | 089/200 ] loss = 0.68862, acc = 0.73003
[ Train | 090/200 ] loss = 0.68398, acc = 0.72439
[ Train | 091/200 ] loss = 0.67086, acc = 0.73741
[ Train | 092/200 ] loss = 0.70456, acc = 0.71311
[ Train | 093/200 ] loss = 0.68567, acc = 0.72786
[ Train | 094/200 ] loss = 0.67503, acc = 0.72830
[ Train | 095/200 ] loss = 0.68981, acc = 0.72613
[ Train | 096/200 ] loss = 0.69931, acc = 0.71267
[ Train | 097/200 ] loss = 0.70451, acc = 0.71528
[ Train | 098/200 ] loss = 0.70992, acc = 0.73003
[ Train | 099/200 ] loss = 0.69446, acc = 0.72005
[ Train | 100/200 ] loss = 0.68985, acc = 0.72092
[ Train | 101/200 ] loss = 0.66479, acc = 0.74392
[ Train | 102/200 ] loss = 0.66749, acc = 0.73090
[ Train | 103/200 ] loss = 0.64942, acc = 0.75174
[ Train | 104/200 ] loss = 0.64894, acc = 0.74783
[ Train | 105/200 ] loss = 0.65479, acc = 0.73481
[ Train | 106/200 ] loss = 0.66659, acc = 0.73003
[ Train | 107/200 ] loss = 0.63883, acc = 0.75000
[ Train | 108/200 ] loss = 0.65636, acc = 0.73047
[ Train | 109/200 ] loss = 0.65493, acc = 0.72526
[ Train | 110/200 ] loss = 0.64205, acc = 0.75000
[ Train | 111/200 ] loss = 0.64493, acc = 0.74566
[ Train | 112/200 ] loss = 0.66218, acc = 0.72613
[ Train | 113/200 ] loss = 0.64778, acc = 0.74609
[ Train | 114/200 ] loss = 0.62781, acc = 0.75651
[ Train | 115/200 ] loss = 0.63063, acc = 0.75477
[ Train | 116/200 ] loss = 0.64340, acc = 0.74479
[ Train | 117/200 ] loss = 0.64218, acc = 0.75043
[ Train | 118/200 ] loss = 0.63747, acc = 0.75347
[ Train | 119/200 ] loss = 0.62402, acc = 0.74740
[ Train | 120/200 ] loss = 0.63123, acc = 0.74523
[ Train | 121/200 ] loss = 0.63149, acc = 0.75130
```

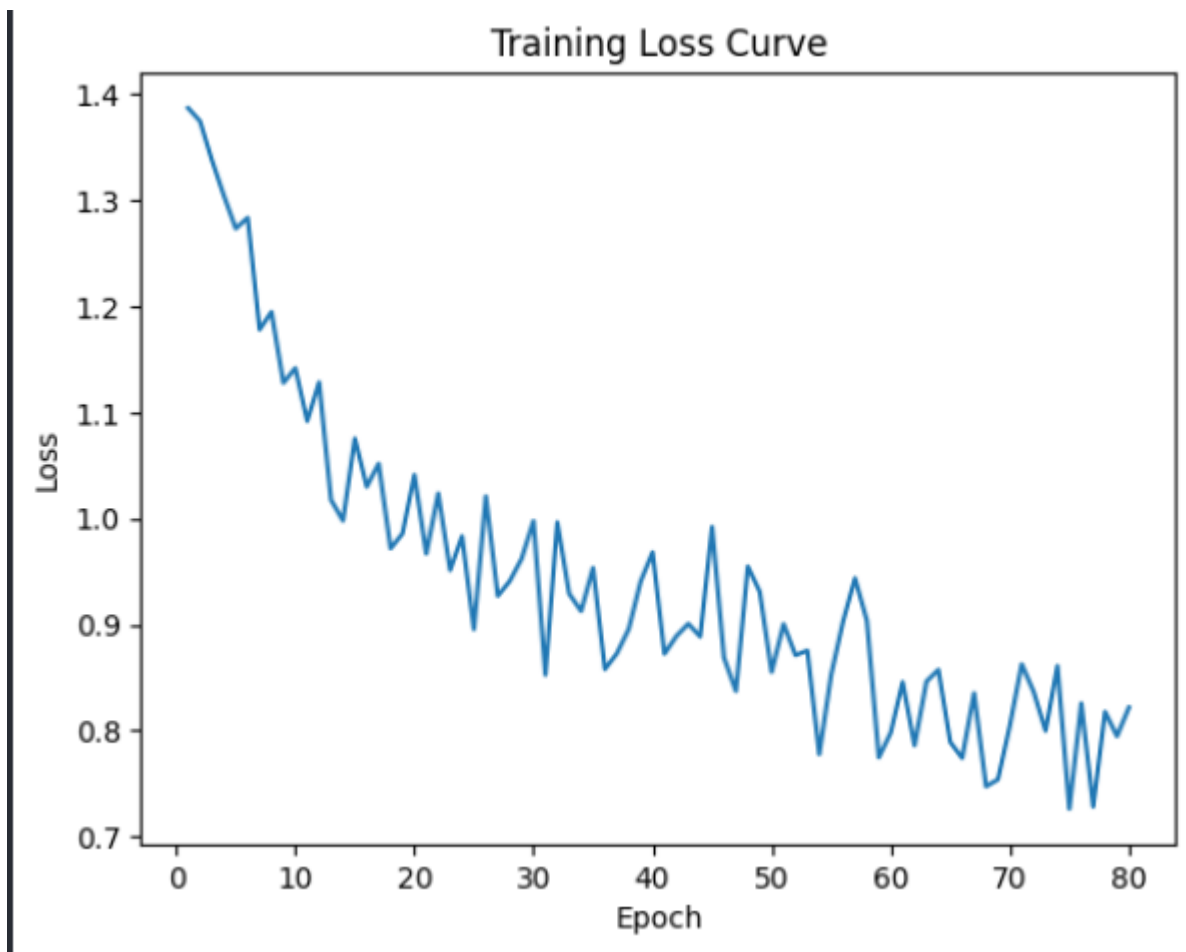
```
[ Train | 122/200 ] loss = 0.63442, acc = 0.74696
[ Train | 123/200 ] loss = 0.62959, acc = 0.74696
[ Train | 124/200 ] loss = 0.62860, acc = 0.75738
[ Train | 125/200 ] loss = 0.60214, acc = 0.76997
[ Train | 126/200 ] loss = 0.63555, acc = 0.75564
[ Train | 127/200 ] loss = 0.61382, acc = 0.75608
[ Train | 128/200 ] loss = 0.60574, acc = 0.76953
[ Train | 129/200 ] loss = 0.63353, acc = 0.75043
[ Train | 130/200 ] loss = 0.61269, acc = 0.76345
[ Train | 131/200 ] loss = 0.60482, acc = 0.76866
[ Train | 132/200 ] loss = 0.62053, acc = 0.75043
[ Train | 133/200 ] loss = 0.62199, acc = 0.75911
[ Train | 134/200 ] loss = 0.61840, acc = 0.75955
[ Train | 135/200 ] loss = 0.61253, acc = 0.75998
[ Train | 136/200 ] loss = 0.59063, acc = 0.76345
[ Train | 137/200 ] loss = 0.60267, acc = 0.76953
[ Train | 138/200 ] loss = 0.61247, acc = 0.75651
[ Train | 139/200 ] loss = 0.59397, acc = 0.76432
[ Train | 140/200 ] loss = 0.59870, acc = 0.75260
[ Train | 141/200 ] loss = 0.61159, acc = 0.75477
[ Train | 142/200 ] loss = 0.59502, acc = 0.77083
[ Train | 143/200 ] loss = 0.61532, acc = 0.76128
[ Train | 144/200 ] loss = 0.59375, acc = 0.76649
[ Train | 145/200 ] loss = 0.60046, acc = 0.75998
[ Train | 146/200 ] loss = 0.60213, acc = 0.76302
[ Train | 147/200 ] loss = 0.59394, acc = 0.76866
[ Train | 148/200 ] loss = 0.60348, acc = 0.75174
[ Train | 149/200 ] loss = 0.58841, acc = 0.76606
[ Train | 150/200 ] loss = 0.59163, acc = 0.76476
[ Train | 151/200 ] loss = 0.59604, acc = 0.77127
[ Train | 152/200 ] loss = 0.56978, acc = 0.78125
[ Train | 153/200 ] loss = 0.59265, acc = 0.77083
[ Train | 154/200 ] loss = 0.58562, acc = 0.76910
[ Train | 155/200 ] loss = 0.58717, acc = 0.77344
[ Train | 156/200 ] loss = 0.58593, acc = 0.76563
[ Train | 157/200 ] loss = 0.58976, acc = 0.75868
```

```
[ Train | 158/200 ] loss = 0.57243, acc = 0.78342
[ Train | 159/200 ] loss = 0.57691, acc = 0.77431
[ Train | 160/200 ] loss = 0.57918, acc = 0.77474
[ Train | 161/200 ] loss = 0.58790, acc = 0.77127
[ Train | 162/200 ] loss = 0.58898, acc = 0.76432
[ Train | 163/200 ] loss = 0.56415, acc = 0.78429
[ Train | 164/200 ] loss = 0.56561, acc = 0.76997
[ Train | 165/200 ] loss = 0.58107, acc = 0.77170
[ Train | 166/200 ] loss = 0.58736, acc = 0.76389
[ Train | 167/200 ] loss = 0.58356, acc = 0.76259
[ Train | 168/200 ] loss = 0.55578, acc = 0.77431
[ Train | 169/200 ] loss = 0.56540, acc = 0.78082
[ Train | 170/200 ] loss = 0.57907, acc = 0.77431
[ Train | 171/200 ] loss = 0.56844, acc = 0.77648
[ Train | 172/200 ] loss = 0.56925, acc = 0.77778
[ Train | 173/200 ] loss = 0.54661, acc = 0.78733
[ Train | 174/200 ] loss = 0.56374, acc = 0.77604
[ Train | 175/200 ] loss = 0.56038, acc = 0.77214
[ Train | 176/200 ] loss = 0.58100, acc = 0.77865
[ Train | 177/200 ] loss = 0.58445, acc = 0.76302
[ Train | 178/200 ] loss = 0.54925, acc = 0.78733
[ Train | 179/200 ] loss = 0.56399, acc = 0.78906
[ Train | 180/200 ] loss = 0.55333, acc = 0.78168
[ Train | 181/200 ] loss = 0.56169, acc = 0.78125
[ Train | 182/200 ] loss = 0.56395, acc = 0.77170
[ Train | 183/200 ] loss = 0.53213, acc = 0.79210
[ Train | 184/200 ] loss = 0.57569, acc = 0.78299
[ Train | 185/200 ] loss = 0.56524, acc = 0.77083
[ Train | 186/200 ] loss = 0.54888, acc = 0.78255
[ Train | 187/200 ] loss = 0.54233, acc = 0.78733
[ Train | 188/200 ] loss = 0.56598, acc = 0.77951
[ Train | 189/200 ] loss = 0.53411, acc = 0.80208
[ Train | 190/200 ] loss = 0.53626, acc = 0.79514
[ Train | 191/200 ] loss = 0.54478, acc = 0.78733
[ Train | 192/200 ] loss = 0.53919, acc = 0.78385
[ Train | 193/200 ] loss = 0.59342, acc = 0.76432
```

```
[ Train | 194/200 ] loss = 0.55459, acc = 0.78212
[ Train | 195/200 ] loss = 0.55791, acc = 0.78733
[ Train | 196/200 ] loss = 0.56041, acc = 0.77257
[ Train | 197/200 ] loss = 0.56144, acc = 0.77431
[ Train | 198/200 ] loss = 0.53288, acc = 0.79123
[ Train | 199/200 ] loss = 0.55022, acc = 0.78212
[ Train | 200/200 ] loss = 0.54890, acc = 0.78819
```

The training method is SD
Average Test Loss: 1.0975
Accuracy: 60.11%

leave-one-subject-out (LOSO)



#LOSO

[Train 001/080]	loss = 1.39148,	acc = 0.26984
[Train 002/080]	loss = 1.36968,	acc = 0.30729
[Train 003/080]	loss = 1.34554,	acc = 0.33755
[Train 004/080]	loss = 1.31836,	acc = 0.36905
[Train 005/080]	loss = 1.28255,	acc = 0.39955
[Train 006/080]	loss = 1.24177,	acc = 0.43998
[Train 007/080]	loss = 1.19213,	acc = 0.46528
[Train 008/080]	loss = 1.16775,	acc = 0.47619
[Train 009/080]	loss = 1.13810,	acc = 0.48810
[Train 010/080]	loss = 1.11452,	acc = 0.50719
[Train 011/080]	loss = 1.09805,	acc = 0.52307
[Train 012/080]	loss = 1.08745,	acc = 0.52406
[Train 013/080]	loss = 1.06122,	acc = 0.54191
[Train 014/080]	loss = 1.05845,	acc = 0.53894
[Train 015/080]	loss = 1.05209,	acc = 0.53894
[Train 016/080]	loss = 1.03720,	acc = 0.55308
[Train 017/080]	loss = 1.01730,	acc = 0.56424
[Train 018/080]	loss = 1.00180,	acc = 0.57564
[Train 019/080]	loss = 1.00428,	acc = 0.56300
[Train 020/080]	loss = 1.00678,	acc = 0.57366
[Train 021/080]	loss = 0.98543,	acc = 0.57961
[Train 022/080]	loss = 0.98117,	acc = 0.58457
[Train 023/080]	loss = 0.96469,	acc = 0.59623
[Train 024/080]	loss = 0.97794,	acc = 0.58209
[Train 025/080]	loss = 0.96610,	acc = 0.59623
[Train 026/080]	loss = 0.95630,	acc = 0.58978
[Train 027/080]	loss = 0.94681,	acc = 0.59573
[Train 028/080]	loss = 0.95551,	acc = 0.59598
[Train 029/080]	loss = 0.94464,	acc = 0.59747
[Train 030/080]	loss = 0.94331,	acc = 0.60045
[Train 031/080]	loss = 0.93377,	acc = 0.60962
[Train 032/080]	loss = 0.91713,	acc = 0.61806
[Train 033/080]	loss = 0.92439,	acc = 0.60838
[Train 034/080]	loss = 0.92899,	acc = 0.61062

```
[ Train | 035/080 ] loss = 0.93475, acc = 0.60913
[ Train | 036/080 ] loss = 0.89145, acc = 0.62872
[ Train | 037/080 ] loss = 0.90030, acc = 0.62922
[ Train | 038/080 ] loss = 0.89666, acc = 0.62426
[ Train | 039/080 ] loss = 0.90135, acc = 0.62500
[ Train | 040/080 ] loss = 0.89093, acc = 0.63318
[ Train | 041/080 ] loss = 0.88395, acc = 0.63318
[ Train | 042/080 ] loss = 0.86790, acc = 0.64335
[ Train | 043/080 ] loss = 0.88375, acc = 0.64236
[ Train | 044/080 ] loss = 0.86716, acc = 0.64261
[ Train | 045/080 ] loss = 0.86914, acc = 0.64261
[ Train | 046/080 ] loss = 0.85908, acc = 0.63591
[ Train | 047/080 ] loss = 0.85600, acc = 0.64732
[ Train | 048/080 ] loss = 0.86407, acc = 0.63864
[ Train | 049/080 ] loss = 0.86981, acc = 0.64509
[ Train | 050/080 ] loss = 0.85537, acc = 0.65005
[ Train | 051/080 ] loss = 0.85261, acc = 0.64955
[ Train | 052/080 ] loss = 0.84570, acc = 0.64683
[ Train | 053/080 ] loss = 0.85018, acc = 0.64559
[ Train | 054/080 ] loss = 0.83311, acc = 0.65799
[ Train | 055/080 ] loss = 0.84264, acc = 0.65699
[ Train | 056/080 ] loss = 0.83072, acc = 0.66047
[ Train | 057/080 ] loss = 0.84846, acc = 0.64931
[ Train | 058/080 ] loss = 0.83914, acc = 0.65675
[ Train | 059/080 ] loss = 0.83497, acc = 0.66369
[ Train | 060/080 ] loss = 0.83220, acc = 0.65675
[ Train | 061/080 ] loss = 0.82188, acc = 0.66964
[ Train | 062/080 ] loss = 0.82009, acc = 0.66096
[ Train | 063/080 ] loss = 0.81117, acc = 0.66592
[ Train | 064/080 ] loss = 0.81282, acc = 0.66964
[ Train | 065/080 ] loss = 0.81657, acc = 0.66691
[ Train | 066/080 ] loss = 0.82328, acc = 0.66394
[ Train | 067/080 ] loss = 0.80010, acc = 0.67088
[ Train | 068/080 ] loss = 0.80955, acc = 0.66865
[ Train | 069/080 ] loss = 0.80075, acc = 0.67634
[ Train | 070/080 ] loss = 0.80046, acc = 0.67684
```

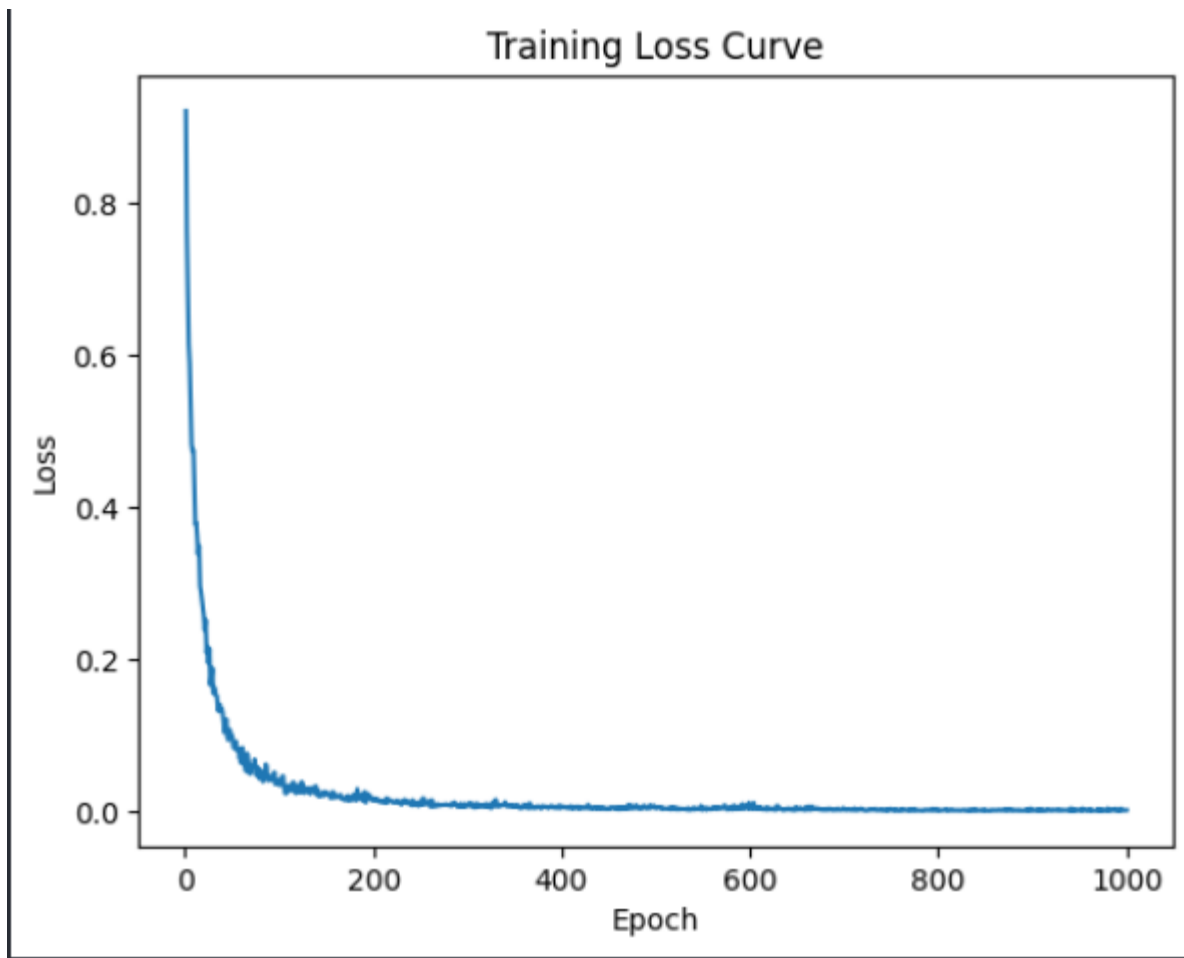
```
[ Train | 071/080 ] loss = 0.78769, acc = 0.68750
[ Train | 072/080 ] loss = 0.79227, acc = 0.66766
[ Train | 073/080 ] loss = 0.80894, acc = 0.67684
[ Train | 074/080 ] loss = 0.78987, acc = 0.67659
[ Train | 075/080 ] loss = 0.79834, acc = 0.67411
[ Train | 076/080 ] loss = 0.79882, acc = 0.67064
[ Train | 077/080 ] loss = 0.79223, acc = 0.67882
[ Train | 078/080 ] loss = 0.78055, acc = 0.67832
[ Train | 079/080 ] loss = 0.79440, acc = 0.67584
[ Train | 080/080 ] loss = 0.79224, acc = 0.66964
```

The training method is LOSO

Average Test Loss: 1.0573

Accuracy: 63.19%

LOSO with finetune



```
[ Train | 001/1000 ] loss = 0.92054, acc = 0.64583
[ Train | 011/1000 ] loss = 0.37803, acc = 0.87153
[ Train | 021/1000 ] loss = 0.23824, acc = 0.93750
[ Train | 031/1000 ] loss = 0.16324, acc = 0.97222
[ Train | 041/1000 ] loss = 0.11888, acc = 0.98611
[ Train | 051/1000 ] loss = 0.08912, acc = 0.98611
[ Train | 061/1000 ] loss = 0.08376, acc = 0.99653
[ Train | 071/1000 ] loss = 0.05901, acc = 0.99653
[ Train | 081/1000 ] loss = 0.04715, acc = 0.99653
[ Train | 091/1000 ] loss = 0.04226, acc = 0.99653
[ Train | 101/1000 ] loss = 0.03412, acc = 0.99653
[ Train | 111/1000 ] loss = 0.02686, acc = 1.00000
[ Train | 121/1000 ] loss = 0.02591, acc = 1.00000
[ Train | 131/1000 ] loss = 0.02982, acc = 0.99653
```

```
[ Train | 141/1000 ] loss = 0.02815, acc = 0.99306
[ Train | 151/1000 ] loss = 0.02468, acc = 1.00000
[ Train | 161/1000 ] loss = 0.01924, acc = 1.00000
[ Train | 171/1000 ] loss = 0.01825, acc = 1.00000
[ Train | 181/1000 ] loss = 0.01687, acc = 1.00000
[ Train | 191/1000 ] loss = 0.02588, acc = 0.99653
[ Train | 201/1000 ] loss = 0.01528, acc = 0.99653
[ Train | 211/1000 ] loss = 0.01373, acc = 1.00000
[ Train | 221/1000 ] loss = 0.01161, acc = 1.00000
[ Train | 231/1000 ] loss = 0.01183, acc = 1.00000
[ Train | 241/1000 ] loss = 0.00795, acc = 1.00000
[ Train | 251/1000 ] loss = 0.01294, acc = 1.00000
[ Train | 261/1000 ] loss = 0.00613, acc = 1.00000
[ Train | 271/1000 ] loss = 0.00886, acc = 1.00000
[ Train | 281/1000 ] loss = 0.00814, acc = 1.00000
[ Train | 291/1000 ] loss = 0.00700, acc = 1.00000
[ Train | 301/1000 ] loss = 0.00892, acc = 1.00000
[ Train | 311/1000 ] loss = 0.00668, acc = 1.00000
[ Train | 321/1000 ] loss = 0.00677, acc = 1.00000
[ Train | 331/1000 ] loss = 0.00749, acc = 1.00000
[ Train | 341/1000 ] loss = 0.00907, acc = 1.00000
[ Train | 351/1000 ] loss = 0.00658, acc = 1.00000
[ Train | 361/1000 ] loss = 0.00585, acc = 1.00000
[ Train | 371/1000 ] loss = 0.00489, acc = 1.00000
[ Train | 381/1000 ] loss = 0.00601, acc = 1.00000
[ Train | 391/1000 ] loss = 0.00620, acc = 1.00000
[ Train | 401/1000 ] loss = 0.00580, acc = 1.00000
[ Train | 411/1000 ] loss = 0.00492, acc = 1.00000
[ Train | 421/1000 ] loss = 0.00460, acc = 1.00000
[ Train | 431/1000 ] loss = 0.00424, acc = 1.00000
[ Train | 441/1000 ] loss = 0.00418, acc = 1.00000
[ Train | 451/1000 ] loss = 0.00317, acc = 1.00000
[ Train | 461/1000 ] loss = 0.00518, acc = 1.00000
[ Train | 471/1000 ] loss = 0.00895, acc = 0.99653
[ Train | 481/1000 ] loss = 0.00403, acc = 1.00000
[ Train | 491/1000 ] loss = 0.00404, acc = 1.00000
```

```
[ Train | 501/1000 ] loss = 0.00534, acc = 1.00000
[ Train | 511/1000 ] loss = 0.00404, acc = 1.00000
[ Train | 521/1000 ] loss = 0.00405, acc = 1.00000
[ Train | 531/1000 ] loss = 0.00340, acc = 1.00000
[ Train | 541/1000 ] loss = 0.00262, acc = 1.00000
[ Train | 551/1000 ] loss = 0.00398, acc = 1.00000
[ Train | 561/1000 ] loss = 0.00601, acc = 1.00000
[ Train | 571/1000 ] loss = 0.00425, acc = 1.00000
[ Train | 581/1000 ] loss = 0.00256, acc = 1.00000
[ Train | 591/1000 ] loss = 0.00793, acc = 0.99653
[ Train | 601/1000 ] loss = 0.00399, acc = 1.00000
[ Train | 611/1000 ] loss = 0.00409, acc = 1.00000
[ Train | 621/1000 ] loss = 0.00425, acc = 1.00000
[ Train | 631/1000 ] loss = 0.00449, acc = 1.00000
[ Train | 641/1000 ] loss = 0.00266, acc = 1.00000
[ Train | 651/1000 ] loss = 0.00358, acc = 1.00000
[ Train | 661/1000 ] loss = 0.00281, acc = 1.00000
[ Train | 671/1000 ] loss = 0.00424, acc = 1.00000
[ Train | 681/1000 ] loss = 0.00349, acc = 1.00000
[ Train | 691/1000 ] loss = 0.00233, acc = 1.00000
[ Train | 701/1000 ] loss = 0.00228, acc = 1.00000
[ Train | 711/1000 ] loss = 0.00421, acc = 1.00000
[ Train | 721/1000 ] loss = 0.00222, acc = 1.00000
[ Train | 731/1000 ] loss = 0.00272, acc = 1.00000
[ Train | 741/1000 ] loss = 0.00234, acc = 1.00000
[ Train | 751/1000 ] loss = 0.00228, acc = 1.00000
[ Train | 761/1000 ] loss = 0.00180, acc = 1.00000
[ Train | 771/1000 ] loss = 0.00128, acc = 1.00000
[ Train | 781/1000 ] loss = 0.00124, acc = 1.00000
[ Train | 791/1000 ] loss = 0.00166, acc = 1.00000
[ Train | 801/1000 ] loss = 0.00218, acc = 1.00000
[ Train | 811/1000 ] loss = 0.00161, acc = 1.00000
[ Train | 821/1000 ] loss = 0.00140, acc = 1.00000
[ Train | 831/1000 ] loss = 0.00105, acc = 1.00000
[ Train | 841/1000 ] loss = 0.00158, acc = 1.00000
[ Train | 851/1000 ] loss = 0.00190, acc = 1.00000
```

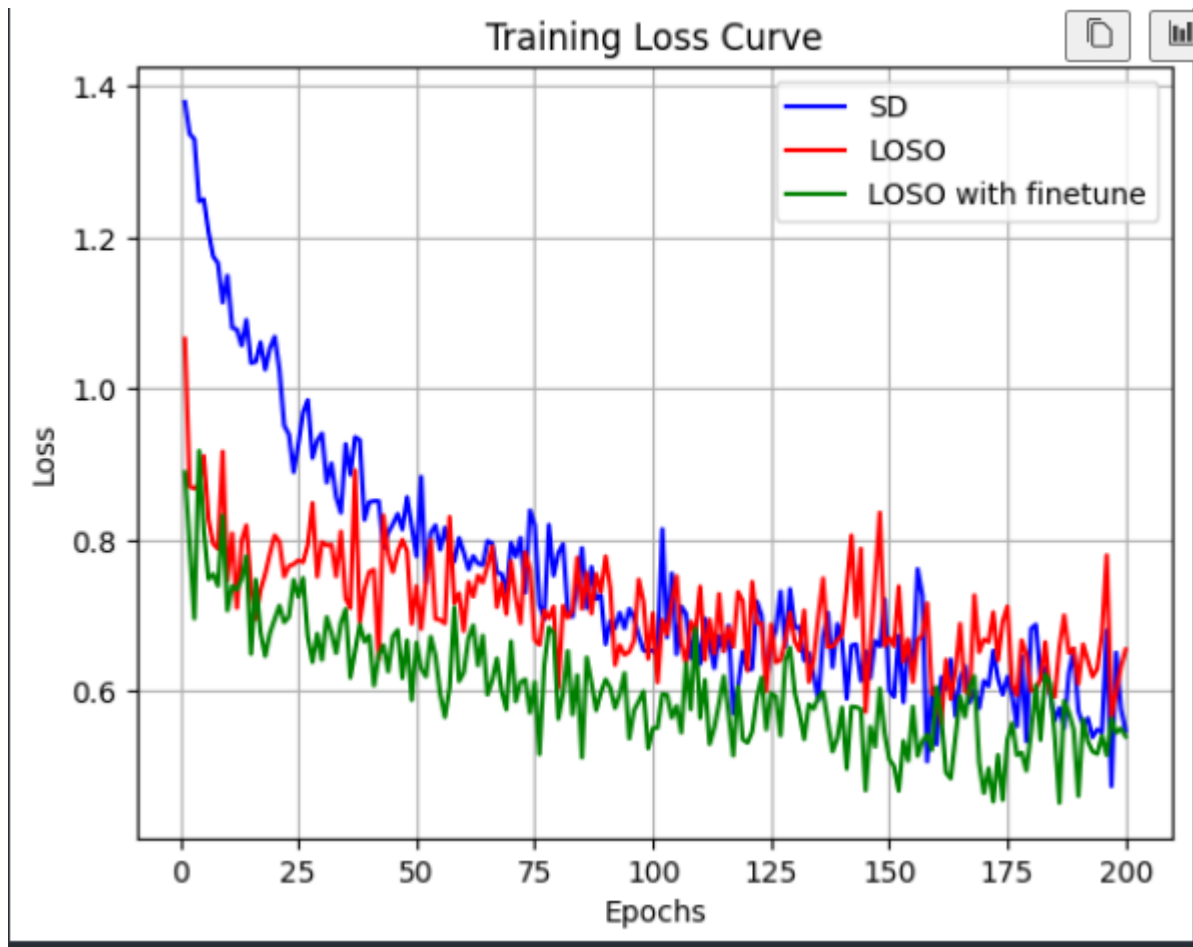
```
[ Train | 861/1000 ] loss = 0.00135, acc = 1.00000
[ Train | 871/1000 ] loss = 0.00347, acc = 1.00000
[ Train | 881/1000 ] loss = 0.00199, acc = 1.00000
[ Train | 891/1000 ] loss = 0.00142, acc = 1.00000
[ Train | 901/1000 ] loss = 0.00165, acc = 1.00000
[ Train | 911/1000 ] loss = 0.00191, acc = 1.00000
[ Train | 921/1000 ] loss = 0.00169, acc = 1.00000
[ Train | 931/1000 ] loss = 0.00214, acc = 1.00000
[ Train | 941/1000 ] loss = 0.00447, acc = 1.00000
[ Train | 951/1000 ] loss = 0.00291, acc = 1.00000
[ Train | 961/1000 ] loss = 0.00124, acc = 1.00000
[ Train | 971/1000 ] loss = 0.00108, acc = 1.00000
[ Train | 981/1000 ] loss = 0.00119, acc = 1.00000
[ Train | 991/1000 ] loss = 0.00146, acc = 1.00000
```

The training method is LOSO with finetune

- After several rounds of fine-tuning, the accuracy can reach 80%.

The loss curve between three method :

- The SD loss dramatically decrease at the begining
- The LOSO with fine-tuning yields significantly better results.



C. Conclusion

- **LOSO with Finetuning:**
 - **Performance:** Best
 - **Advantages:**
 - Utilizes almost samples from the dataset for training and then fine-tunes the model, leading to enhanced performance on the test set.
 - Demonstrates improved generalization ability across participants.
 - **Use Case:** Ideal for scenarios where participant-specific data is limited, as it significantly boosts model performance.
- **LOSO (Leave-One-Subject-Out):**

- **Performance:** Intermediate
- **Advantages:**
 - Trains the model using data from all participants except the one being tested, leveraging the overall dataset.
 - No additional fine-tuning required, simplifying the training process.
- **Use Case:** Suitable for participant-independent scenarios, but may perform slightly worse compared to LOSO with Finetuning.
- **SD (Subject-Dependent):**
 - **Performance:** Worst
 - **Advantages:**
 - The model is trained solely on data from a specific participant, which may yield good performance for that participant's test data.
 - **Disadvantages:**
 - Poor generalization ability, making it less effective on data from unseen participants.
 - **Use Case:** Best for deep optimization on a single participant's data, but less effective for cross-participant applications.

Summary:

- **LOSO with Finetuning** offers the best performance, especially in scenarios requiring high generalization and fine-tuning.
- **LOSO** provides better generalization compared to **SD**, but not as effective as LOSO with Finetuning.
- **SD** may perform well for specific participants but lacks robustness in cross-participant scenarios.

4. Discussion

A. What is the reason to make the task hard to achieve high accuracy?

- **Individual Differences:** The EEG signals vary significantly between different individuals, leading to poor generalization performance of the model.
- **Important Feature Omission:** The key features relevant to the task were not extracted.
- **Insufficient Sample Size:** The amount of training data is not enough for the model to learn adequately.
- The model will overfitting

B. What can you do to improve the accuracy of this task?

- Implement regularization techniques, such as dropout and L2 regularization, to mitigate overfitting.
- Implement data normalization to reduce differences between individuals.
- I tried some data augmentation, but the results were not satisfactory.

C. Do some data augmentation (Compare with origin one)

- We can see that it doesn't exhibit significantly better performance when we add some noise randomly

```
def normalize(self, feature):  
  
    mean = torch.mean(feature, dim=1, keepdims=True)  
    std = torch.std(feature, dim=1, keepdims=True)  
    feature = (feature - mean) / (std + 1e-7) # Adding a small value to avoid division by zero  
  
    # add some noise  
    if np.random.rand() > 0.5 :  
        noise = torch.randn_like(feature) * 0.1 # 标准差为0.1的高斯噪声  
        feature = feature + noise  
  
    return feature
```

